

FEATURE SPEC

Recurring tasks feature spec for the portal

Implementation spec for the recurring-tasks feature already shipped on the dev build (nikkotrinidad-cpu.github.io/flizow). Walks through the user-visible behaviour, the data shape, the spawn-on-Done hook, the picker UI, and the build checklist for porting into the portal. Every reference file in the dev build is named at the end so the portal dev can read the implementation directly.

This document is interactive. Tick each implementation step in the build checklist as you ship it, then save the PDF. State persists so we can keep one source of truth as the portal build catches up.

Overview

A new field on every kanban card called Repeat. The user picks how often the card should loop (Daily / Weekly / Monthly / Yearly / Custom). When the card moves into the Done column, the system automatically spawns a fresh copy back into To Do with the next computed due date. The original card stays in Done as history. The new card carries the same recurrence rule, so the loop continues until the user pauses it, sets an end date, or removes the rule.

How it behaves

What the user does

1. Open any kanban card on a service board or the Ops board.
2. Inside the card modal, find the new Repeat field (sits between Start date and Labels).
3. Click the field. A small popover opens with five preset options: Doesn't repeat (default), Daily, Weekly on , Monthly on the , Yearly, plus a Custom... entry that opens an inline editor.
4. Pick a preset or build a custom rule. Optionally toggle End by (sets a date the rule stops on) or Pause (keeps the rule attached but stops it spawning new cards).
5. The summary line on the field updates immediately ("Weekly on Tue", "Every 2 weeks on Mon, Wed, Fri", etc.).
6. A small loop icon appears on the card tile in the kanban view so anyone glancing at the board knows this card is part of a series.

What happens on Done

When the user drags or moves a recurring card into the Done column, the store immediately spawns a new card back in To Do. The new card has a fresh due date computed from the rule (see Data shape below for how it's calculated), the same title, description, labels, priority, assignee, slots, and checklist (with reset progress). The new card carries the same recurrence rule. The original card stays Done as history; comments and activity stay on it.

What happens when the rule ends

If the rule has an endsAt date and the next computed due date would be after that date, no new card spawns. The card the user just finished stays Done. The rule has effectively ended; the user can edit the rule on a new card or remove it. Same behaviour when Pause is on: no spawn happens, but the rule stays attached so the user can resume later.

UI surfaces

Card modal: the Repeat field

A new meta-row sits between Start date and Labels in the card modal. The field shows a small loop icon, the current rule's summary line (or "Doesn't repeat" when unset), and a chevron indicating the popover opens on click.

Card tile: the loop icon

When a card has a recurrence rule (paused or active), a tiny loop icon (■) appears on the card tile next to the comment and attachment counts. The icon dims when the rule is paused. Hovering shows a tooltip: "Recurring" or "Recurring (paused)". This is the only visual cue on the tile so a teammate dragging the card to Done knows the loop is about to fire.

The popover

Click the Repeat field to open a small popover anchored below the field. Five preset rows on top: Doesn't repeat / Daily / Weekly on / Monthly on the / Yearly / Custom... The active preset shows a filled radio dot. Selecting a preset other than Doesn't repeat applies the rule immediately (no Save button). Picking Doesn't repeat clears the rule and closes the popover.

Custom editor

When Custom is active, the popover expands to show three rows: Every [number] [days/weeks/months/years], plus an On row that adapts to the pattern (day-of-week chips for weekly, day-of-month input for monthly). The number defaults to 1; the user can change interval and pattern independently. Day chips toggle on click; the day-of-month input clamps 1-31.

End by + Pause

Below the presets, two toggles. End by enables an inline date picker that defaults to 3 months from today. Pause keeps the rule attached but stops the spawn-on-Done hook. The summary line on the field reflects both: "Weekly on Tue, ends Aug 7, 2026" and "Paused: Weekly on Tue".

Data + math

Recurrence interface

Add to types/flizow.ts:

```
interface Recurrence {
  pattern: 'daily' | 'weekly' | 'monthly' | 'yearly';
  interval: number;           // every N ; >= 1
  byDay?: number[];          // weekly only; 0=Sun..6=Sat
  byMonthDay?: number;       // monthly only; 1-31
  endsAt?: string;           // optional ISO date
  paused?: boolean;
}
```

Then add the field to Task and OpsTask:

```
interface Task {
  // ...existing fields
  recurrence?: Recurrence;
}
```

Pure functions

Three helpers in utils/recurrence.ts: `nextDueDate(rule, anchorISO)` · returns the next ISO date or null when the rule has ended. The anchor is the previous due date. `summarizeRecurrence(rule)` · returns the plain-language summary line ("Weekly on Tue", "Monthly on the 5th", etc.) used on the field and the card tooltip. `defaultRule(pattern, anchorISO)` · returns a sensible default rule for each preset, seeded from the anchor (so "Weekly on Tuesday" comes pre-filled when the anchor is a Tuesday).

Date math edge cases

Daily: anchor + interval days. Weekly with no byDay: anchor + (7 * interval) days. Weekly with byDay: find the next day in the mask after the anchor's weekday this week; if exhausted, jump interval weeks and pick the first day. Monthly: same day of month next interval months; clamp to last day if shorter (e.g. day 31 in Feb resolves to 28/29). Yearly: same month and day, +interval years. endsAt: when the next computed date is strictly after endsAt, return null. Equal to endsAt is allowed.

Store wiring

Spawn-on-Done hook

In `flizowStore.ts`, hook into `updateTask` after the activity diff/log block. When the card transitions INTO done (before `!= done` && after `== done`) AND has a non-paused recurrence rule, call `spawnRecurringTask(updated)`. Same hook in `updateOpsTask` calling `spawnRecurringOpsTask`. The trigger is entering Done from any other column; the previous column doesn't matter.

Spawn semantics

spawnRecurringTask creates a new Task by spreading the source card and overriding: id (fresh), columnId (todo), dueDate (nextDueDate(rule, source.dueDate)), createdAt (now). Resets blockerReason, flaggedForWip, archived, archivedAt. Maps checklist with fresh ids and done: false. Comments and activity DO NOT carry. Pushes onto data.tasks; appends id to the parent service's taskIds. Logs a 'created' activity entry on the new card: "spawned by recurrence from ". Same shape for spawnRecurringOpsTask against data.opsTasks; ops cards have no parent service so no taskIds maintenance.

Manual duplicate is one-off

duplicateTask and duplicateOpsTask must drop the recurrence field on the copy. A user manually duplicating a card is creating a one-off, not another loop in the series. If they want recurrence on the duplicate, they set it themselves.

Edge cases

Card has no due date when moved to Done. The recurrence rule has no anchor to compute from. Skip the spawn silently. The user can set a due date on the next instance manually if needed; this is rare in practice because recurring cards almost always have a due date by the time they're worked on.

Card is moved out of Done back to a previous column. The previously-spawned next-instance stays where it is. We don't try to roll back the spawn; doing so creates weird-undo edge cases and the user can just delete the spawned card manually if they really meant to undo the completion.

Garbage anchor date. nextDueDate returns null defensively when the anchor doesn't parse as YYYY-MM-DD. The spawn is skipped silently.

Non-positive interval. Treat as 1. Defensive against bad data from old saves or manual JSON edits.

Holidays. v1 does not auto-shift the next computed date when it lands on a holiday. The system has the holidays catalog wired (see Ops > Time off Schedules) but using it here is Phase 2. Acceptable to add a small inline warning in the picker if the next date happens to fall on a holiday, but no auto-fix.

Edits to a card don't propagate. If the user edits the title or any field on one instance, the next spawn uses the rule + the source card AS IT IS. There is no "this and future" dialog. Each spawned card is its own card; edits to it stay local.

Out of scope for v1

Deliberately deferred. Add later only if real usage shows pain.

- True scheduler that fires on a calendar tick. v1 spawns only on Done. If a card sits unfinished for a month, no fresh one appears. Notifications panel still flags it as overdue.
- Holiday auto-shift. The Holidays catalog exists but isn't consulted by the recurrence math. Phase 2.
- "This and future" edit semantics. Edits to a card affect that card only.

- Bulk recurrence editor / cross-board view of all recurring cards. Phase 2 if pain shows up.
- Daily-digest notification when N recurring cards spawn. Existing overdue / due-today notifications cover the value.

Implementation checklist

Tick each step as you ship. Severity tags (HIGH / MED) flag what blocks the feature working at all vs. what's polish.

- ☐ **HIGH** Add Recurrence interface and recurrence?: Recurrence field on Task and OpsTask in types/flizow.ts.
- ☐ **HIGH** Build pure functions in utils/recurrence.ts: nextDueDate, summarizeRecurrence, defaultRule, dayOfWeekLabel.
- ☐ **HIGH** Write unit tests covering daily / weekly with single + multi day mask / weekly mask exhausted / monthly with overflow / monthly across year boundary / yearly / endsAt boundary / garbage anchor / non-positive interval. Aim for ~25-30 test cases.
- ☐ **HIGH** Hook spawn-on-Done into updateTask: when columnId transitions into done and the card has a non-paused recurrence rule, call spawnRecurringTask.
- ☐ **HIGH** Implement spawnRecurringTask: spread source, override id / columnId / dueDate / createdAt, reset blockerReason / flaggedForWip / archived / checklist progress, push onto data.tasks, append to parent service.taskIds, log activity.
- ☐ **HIGH** Same hook + spawn in updateOpsTask / spawnRecurringOpsTask. Ops cards have no parent service so no taskIds maintenance; reset enteredDaysAgo / overrideMod / overrideLabel on the spawn.
- ☐ **MED** Drop the recurrence field on manual duplicateTask and duplicateOpsTask copies (manual copy is one-off).
- ☐ **HIGH** Build the RecurrencePicker component: summary button + popover with 5 presets, Custom editor (interval + pattern + day-of-week chips for weekly / day-of-month input for monthly), End by + Pause toggles. Click-outside-to-close. Plain-language summary line.
- ☐ **HIGH** Wire the Repeat meta-row into the card modal (FlizowCardModal.tsx) between Start date and Labels. Pass the card's dueDate as the anchor (or today's ISO when dueDate is unset, so the preset labels still read correctly).
- ☐ **MED** Add the loop icon (■ stroked, 14x14) on the kanban card tile in BoardPage and OpsPage. Show whenever recurrence is set; dim style when paused. Tooltip: "Recurring" / "Recurring (paused)".
- ☐ **MED** Style the popover for dark mode parity. Use named spacing tokens (--sp-xs, --sp-sm, --sp-md). No em dashes in copy. Make sure the End by + Pause toggle click targets are scoped to the visible label, not the full row width.

Reference files in the dev build

Read these to see how the dev build implements each piece. The portal can mirror the same shape or adapt as needed.

src/types/flizow.ts

Recurrence interface; recurrence field on Task and OpsTask.

src/utils/recurrence.ts

Pure date math + summary formatting. No external deps.

src/test/recurrence.test.ts

29 unit tests covering the math.

src/store/flizowStore.ts

spawnRecurringTask + spawnRecurringOpsTask; hook in updateTask + updateOpsTask; rule-drop in duplicateTask + duplicateOpsTask.

src/components/RecurrencePicker.tsx

Self-contained component: summary button + popover + Custom editor + End by + Pause.

src/components/FlizowCardModal.tsx

Wires RecurrencePicker into the card modal as a meta-row after Start date.

src/pages/BoardPage.tsx

Loop icon (RecurrenceIcon component) added to card tile next to comment count.

src/pages/OpsPage.tsx

Same loop icon on the Ops card tile.

src/styles/flizow.css

Picker + popover + Custom editor + End by + Pause styles. Search for "Recurrence picker" comment block.